

makes `cheese` *refer* to the same list object in the computer's memory as `spam`. It doesn't *duplicate* the list object. This is why changing `spam` also changes `cheese`: both variables refer to the same list object.

The same principle applies to mutable objects passed to a function call. Enter the following into the interactive shell, and note that the global variable `spam` and the local parameter (remember, parameters are variables defined in the function's `def` statement) `theList` both refer to the same object:

```
>>> def printIdOfParam(theList):
...     print(id(theList))
...
>>> eggs = ['cat', 'dog', 'eel']
>>> print(id(eggs))
2356893256136
>>> printIdOfParam(eggs)
2356893256136
```

A visualization of the execution of this code is at <https://autbor.com/listcopygotcha2>. Notice that the identities returned by `id()` for `eggs` and `theList` are the same, meaning these variables refer to the same list object. The `eggs` variable's list object wasn't copied to `theList`; rather, the reference was copied, which is why both variables refer to the same list. A reference is only a few bytes in size, but imagine if Python copied the entire list instead of just the reference. If `eggs` contained a billion items instead of just three, passing it to the `printIdOfParam()` function would require copying this giant list. This would eat up gigabytes of memory just to do a simple function call! That's why Python assignment only copies references and never copies objects.

One way to prevent this gotcha is to make a copy of the list object (not just the reference) with the `copy.copy()` function. Enter the following into the interactive shell:

```
>>> import copy
>>> bacon = [2, 4, 8, 16]
>>> ham = copy.copy(bacon)
>>> id(bacon), id(ham)
(2356896337352, 2356896337480)
>>> bacon[0] = 'CHANGED'
>>> bacon
```
